

Module 6: Spring Data JPA with Spring Boot, Hibernate

1. Spring Data JPA – Quick Example

Codes:

First create SQL Tables

```
CREATE DATABASE ormlearn;
```

```
USE ormlearn;
```

```
CREATE TABLE country (code VARCHAR(2) PRIMARY KEY, name VARCHAR(50));
```

```
INSERT INTO country VALUES ('IN', 'India');
```

```
INSERT INTO country VALUES ('US', 'United States of America');
```

```
SELECT * FROM country;
```

```
exit
```

Output:

```
mysql> SELECT * FROM country;
+-----+-----+
| code | name                |
+-----+-----+
| IN   | India               |
| US   | United States of America |
+-----+-----+
2 rows in set (0.00 sec)
```

Codes:

Country.java

```
package com.cognizant.orm_learn.model;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
@Entity
@Table(name = "country")
public class Country {
    @Id
    @Column(name = "code")
    private String code;
    @Column(name = "name")
    private String name;
    public Country() {
    }
    public String getCode() {
        return code;
    }
}
```

```

    public void setCode(String code) {
        this.code = code;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "]";
    }
}

```

CountryRepository.java

```

package com.cognizant.orm_learn.repository;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.cognizant.orm_learn.model.Country;
@Repository
public interface CountryRepository extends JpaRepository<Country, String> {
}

```

CountryService.java

```

package com.cognizant.orm_learn.service;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.cognizant.orm_learn.model.Country;
import com.cognizant.orm_learn.repository.CountryRepository;
import jakarta.transaction.Transactional;
@Service
public class CountryService {
    @Autowired
    private CountryRepository countryRepository;
    @Transactional
    public List<Country> getAllCountries() {
        return countryRepository.findAll();
    }
}

```

OrmLearnApplication.java

```

package com.cognizant.orm_learn;
import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;

```

```

import com.cognizant.orm_learn.model.Country;
import com.cognizant.orm_learn.service.CountryService;
@SpringBootApplication
public class OrmLearnApplication {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(OrmLearnApplication.class);
    private static CountryService countryService;
    public static void main(String[] args) {
        ApplicationContext context =
            SpringApplication.run(OrmLearnApplication.class, args);
        countryService = context.getBean(CountryService.class);
        LOGGER.info("Inside main");
        testGetAllCountries();
    }
    private static void testGetAllCountries() {
        LOGGER.info("Start");
        List<Country> countries =
            countryService.getAllCountries();
        LOGGER.debug("Countries={}", countries);
        LOGGER.info("End");
    }
}

```

Output

```

26-07-26 00:26:17.329 restartedMain DEBUG org.hibernate.SQL          logStatement 126 select c1_0.code,c1_0.name from country c1_0
26-07-26 00:26:17.387 restartedMain DEBUG c.c.o.OrmLearnApplication testGetAllCountries 42 Countries=[Country [code-IN, name-India], Country [code=US, name=United States of America]]
26-07-26 00:26:17.388 restartedMain INFO c.c.o.OrmLearnApplication testGetAllCountries 44 End
26-07-26 00:26:17.396 locationShutdownHook INFO r.EntityManagerFactoryBean destroy 693 Closing JPA EntityManagerFactory for persistence unit 'default'
26-07-26 00:26:17.400 locationShutdownHook INFO c.z.h.HikariDataSource close 349 HikariPool-1 - Shutdown initiated...
26-07-26 00:26:17.411 locationShutdownHook INFO c.z.h.HikariDataSource close 351 HikariPool-1 - Shutdown completed.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 8.895 s
[INFO] Finished at: 2026-07-26T00:26:17+05:30
[INFO] -----

```

2. Difference between JPA, Hibernate and Spring Data JPA

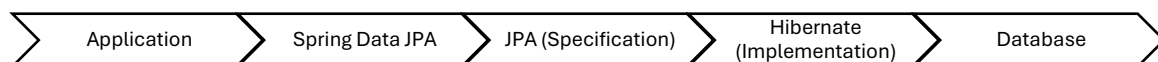
Java Persistence API (JPA) is a Java specification (JSR 338) used to persist, retrieve, update, and delete data from relational databases using Java objects. JPA only defines standards and APIs; it does not provide an actual implementation. To use JPA, an implementation such as Hibernate is required.

Hibernate is an Object Relational Mapping (ORM) framework that implements the JPA specification. It converts Java objects into database tables and SQL queries automatically, reducing the need to write SQL manually. Hibernate provides features such as caching, lazy loading, transaction management, and database portability.

Spring Data JPA is a Spring framework module built on top of JPA. It does not implement JPA itself but simplifies database operations by reducing boilerplate code. It automatically provides CRUD operations, repository implementations, pagination, sorting, and transaction management.

Feature	JPA	Hibernate	Spring Data JPA
Definition	A Java specification (API) for Object Relational Mapping (ORM).	An ORM framework that implements JPA.	A Spring module that simplifies JPA-based data access.
Type	Specification	Framework	Spring Framework Module
Developed By	Oracle (Java Community Process)	Hibernate Team (Red Hat)	Spring Team
Purpose	Defines standards for ORM.	Implements ORM functionality and JPA specification.	Simplifies database access using JPA repositories.
SQL Handling	Does not generate SQL by itself.	Generates SQL automatically.	Uses JPA/Hibernate to generate SQL.
CRUD Operations	Must be written using EntityManager.	Uses Session or JPA EntityManager.	Provides built-in CRUD methods through JpaRepository.
Boilerplate Code	More	Moderate	Very Less
Query Support	JPQL	HQL, JPQL, Native SQL	Derived Query Methods, JPQL, Native SQL
Dependency	Requires an implementation such as Hibernate.	Can work independently or as a JPA provider.	Depends on both Spring Framework and a JPA provider like Hibernate.
Ease of Use	Moderate	Easy	Very Easy

Relationship between JPA, Hibernate, and Spring Data JPA



Conclusion

- **JPA** defines the standard rules for Object Relational Mapping (ORM) but does not provide an implementation.
- **Hibernate** is a popular ORM framework that implements the JPA specification and handles communication with the database.
- **Spring Data JPA** is built on top of JPA and further simplifies data access by automatically providing repository implementations and CRUD operations, resulting in cleaner and more maintainable code.